2026 · 码道 AI 编程大赛　参赛作品

缓 — 缓

个人学术工作台

安排任务、管理计划；用一个能关联待办的番茄钟留出专注时间；

把教材拆成章、节，逐节读讲义、看原文、向 AI 提问；

思绪混乱的时候可以自然地聊，等你确认之后，才把有用的部分变成待办。

项目	内容
作品形态	网页版 / Windows 桌面版 / 安卓 App（三端独立打包）
技术栈	Node.js 内置模块 + 原生 JavaScript（无框架、无构建） Electron 45.2.3 · Capacitor 8 · PDF.js 4.10
代码规模	自研 37,366 行（服务端 16,270 / 前端 20,106 / 桌面壳 990）
自测	765 项断言（接口 388 / AI 55 / 前端 322）
交付物	Windows 便携版与安装版、安卓 APK、源码包、演示脚本与过程记录
文档版本	v1.0　2026 年 11 月 10 日

本作品的全部数据只保存在本机。没有账号体系，不上传，不连接任何云服务。

目　录

一　场景背景：四个工具，四份记忆 3
1.1 一个具体的学习日 3
1.2 断点具体在哪里 3
1.3 这个作品要回答的三个问题 3
1.4 刻意不做的事 4
二　功能架构：一套代码，三种外壳 5
2.1 三种外壳 6
2.2 服务端的分层与硬约束 6
2.3 数据长什么样 6
2.4 五个模块的实际界面 7
2.5 一句话怎么变成一条待办 10
三　核心亮点：六个不那么显眼、但决定成败的设计 11
3.1 番茄钟绑在一条具体的待办上 11
3.2 教材：拆章、逐节读、就这一节提问 11
3.3 两段式确认：先梳理，再行动 11
3.4 模型自己配：多供应商、多模型、只存本机 11
3.5 浮窗与主程序在同一个进程里 12
3.6 服务端零第三方依赖 12
3.7 三端独立，但把「将来互通」写进了契约 12
四　码道应用说明：先给约束，再让 AI 写，用测试收口 13
4.1 三类典型用法 13
4.2 三个真实案例 13
4.3 沉淀下来的五条方法 14
4.4 明确没有交给 AI 的部分 14
4.5 过程记录在哪 14
五　工程数据与验证结果 15
5.1 代码规模 15
5.2 自测：765 项断言 15
5.3 产物与实测 16
5.4 已知限制 16
六　交付物与运行方式 17
6.1 提交目录 17
6.2 三条运行路径 17
6.3 首次配置模型（约 3 分钟） 18
6.4 数据在哪、怎么备份 18

提示：目录页码由 Word 在打开时刷新；本文件同时提供已刷新页码的 PDF 版本。

一 场景背景：四个工具，四份记忆

准备考研、考证或者跨专业自学的人，一天里大概会打开四五个不同的工具：备忘录里记着「今天要写完第三章」，番茄钟在手机里滴答，教材 PDF 开在阅读器里，遇到不懂的段落复制到聊天窗口问 AI，晚上再写两句话复盘。每一件工具都做得不错，问题出在它们互不相识。

1.1 一个具体的学习日

早上八点，打开待办清单，看见昨天列的六条任务，其中两条已经拖了三天。点开番茄钟，计时二十五分钟——可它不知道这二十五分钟属于哪一门课，结束时只在角落里加了「1 个番茄」。

十点读教材，第二章第三节讲得含糊，想把这一段丢给 AI 问清楚。教材在阅读器里，AI 在浏览器另一个标签页，中间隔着一次复制、一次粘贴、一次「你说的是哪本书」。问完想记下来，又不知道记在哪里：待办清单装不下解释，笔记软件里的这段文字和明天的计划没有关系。

这种断裂不会立刻造成损失，它的代价是延迟出现的：一周之后，你只知道「学了二十六个番茄」，却回答不了「哪一章最费时间」「上次卡在哪里」「当时是怎么想通的」。

1.2 断点具体在哪里

断点	具体表现	带来的后果
计时与任务脱节	计时器只知道「25 分钟」，不知道这 25 分钟属于哪条任务、哪门课。	时长只剩一个总数，无法按内容回看；也没法回答「这本书我到底花了多久」。
阅读与提问分离	教材在阅读器、问题在聊天窗口，来回复制粘贴，还要重复交代背景。	读到一半被打断，多数人索性不问了，问题留在原地。
记录与行动不通	复盘写在笔记里，待办在另一个软件里；AI 也读不到你的待办。	想清楚了，仍要手工再录一遍，多数时候就不录了。
AI 直接改数据	不少助手会「热心地」替你建任务、改计划，事后还要清理。	不敢让它碰真实数据，助手退化成聊天窗口。

1.3 这个作品要回答的三个问题

1. 能不能只在一个地方完成「定计划 → 专注 → 读教材 → 问 AI → 落成任务」，而不是在五个窗口之间搬运？

2. 计时能不能知道自己计的是哪一条待办，结束后按实际经过的时间记账，而不是按计划的二十五分钟？
3. AI 能不能参与整理，但一个字都不擅自写进我的数据里？

「缓一缓」把这三件事做成了一个本地应用：一个 Node 服务端、一套原生前端、三种外壳。它不替你安排一切，只把散落的环节接起来，并且在写入之前一定停下来问一次。

1.4 刻意不做的事

一份作品说明如果只写「做了什么」，很难看出取舍。下面这些是明确不做的，它们和功能列表同样重要：

不做	原因
排行榜、连续打卡、番茄数徽章	把学习变成刷数据，一旦断签就再也不想打开。
云同步与账号体系	教材、对话、错题都属于个人；不需要注册就能用，是这个工具的前提。
内置模型与 API Key	内置 Key 等于把成本和使用者的数据一起交出去；模型由使用者自己填。
AI 自动安排整张计划表	计划的价值在于自己想清楚；AI 只能整理，不能替代判断。
三端数据互通	本期三端各自独立，先把「契约」写进代码，同步留到下一版。

二 功能架构：一套代码，三种外壳

整个应用只有一份服务端、一份前端。桌面版和安卓版不是重写，而是给同一份代码套上不同的外壳：Electron 提供窗口，Capacitor 提供 WebView 与内嵌的 Node 运行时。这样做的代价是要自己处理「本地服务什么时候就绪」，好处是任何一处修改三端同时生效。



图 2-1　功能架构：三种外壳共用同一套服务端与数据层

这张图从上往下读：最上面是三种外壳，中间是同一份前端与同一份服务端，最下面是 data/ 目录。每一层只依赖它下面那一层——界面不直接读文件，服务端不关心自己跑在哪个外壳里，所以换壳不用改业务代码。

2.1 三种外壳

形态	外壳	数据落在哪	怎么启动
网页版	浏览器直接访问，Node 内置 HTTP 服务	项目下 data/	node server/server.js
桌面版	Electron 45.2.3，主进程内可再开一个浮窗	%APPDATA%\huan-yihuan-desktop\data	双击 exe，3–5 秒后就绪
安卓版	Capacitor 8 + @capawesome/capacitor-nodejs 内嵌 Node 18	App 私有目录（app.datadir()/data）	安装 APK，首次 10–20 秒拉起运行时

三端数据目录彼此隔离：桌面版与安卓版各存各的，互不覆盖；同一台电脑上还可以再跑第二份实例（scripts/start-two.js）。

2.2 服务端的分层与硬约束

3.7 万行代码最容易腐烂的地方，是「谁都能直接读写文件」。所以第一件事不是写功能，而是把分层和禁区先钉死，再让代码往里填：

文件	职责	硬约束
server.js	HTTP 路由、参数校验、流程编排、静态文件	不直接读写数据文件，只调用 db / 领域模块
db.js	唯一的持久化入口：读写、软删除、迁移、回收站	其它模块不得自行 fs.writeFile
models.js	字段白名单、默认值、时间戳、id 生成	新字段必须在这里登记，否则存不进去
ai.js / llm.js / tools.js	三个引擎、流式输出、工具调用循环（最多 3 轮）	模型只能提议，不能直接落库
model-config.js	多供应商、多模型、对话级 modelRef、Key 脱敏	Key 不出服务端，接口只回后四位
textbook.js / articles.js / projects.js	教材拆章、外刊批注、学习项目归属	只依赖 db 暴露的接口

2.3 数据长什么样

没有数据库。data/ 下每个条目一个 JSON 文件，文件名就是 id，打开文件夹就能读懂；坏一条只坏一条，备份就是复制目录。

目录 / 文件	内容	为什么这样放
tasks/ plans/ projects/	待办、长期计划、学习项目	一条任务一个文件，删除进 .trash 可找回
conversations/	对话、梳理卡、行动方案卡	方案与消息同文件，确认状态不会分叉
textbooks/ articles/	教材章节与原文、外刊批注与生词	教材 PDF 原文单独存放，正文抽取后仍可回看
focus-sessions/ focus-active.json	已完成的专注记录、唯一一份进行中的计时	网页与浮窗共用同一份计时状态
settings.json	模型供应商、Key、界面偏好	Key 只写本机，界面上只显示后四位
entries/	第一版留下的旧记录（只读保留）	迁移成档案时保留原文件，不做破坏性改写

2.4 五个模块的实际界面

下面六张图都是用应用自己的渲染进程截的真实界面，不是设计稿。



图 2-2　工作台：今天要做什么、这周的时间去哪了、下一步可以进哪

工作台只回答三个问题：今天还剩什么、这一周的时间花在哪里、现在最适合进哪个模块。没有催促文案，也没有「你已连续学习 X 天」这类提醒。

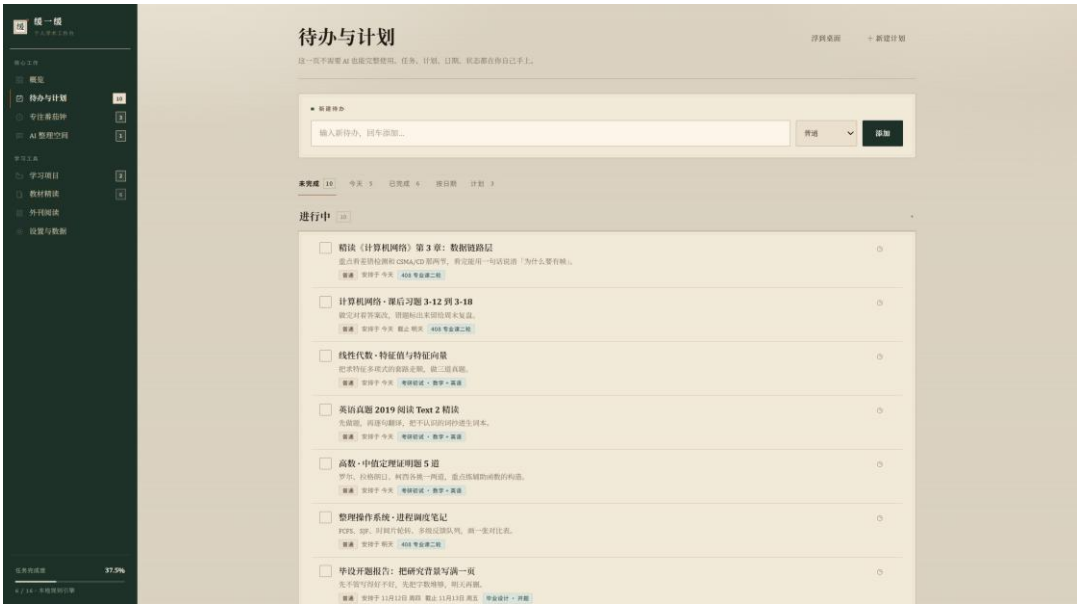


图 2-3　待办与计划：任务可以挂到长期计划与学习项目上

一条任务可以同时属于一个长期计划和一个学习项目，桌面端与安卓端的任务结构完全一致；完成、延期、放弃都留痕，周回顾时能看到「改过几次日期」。

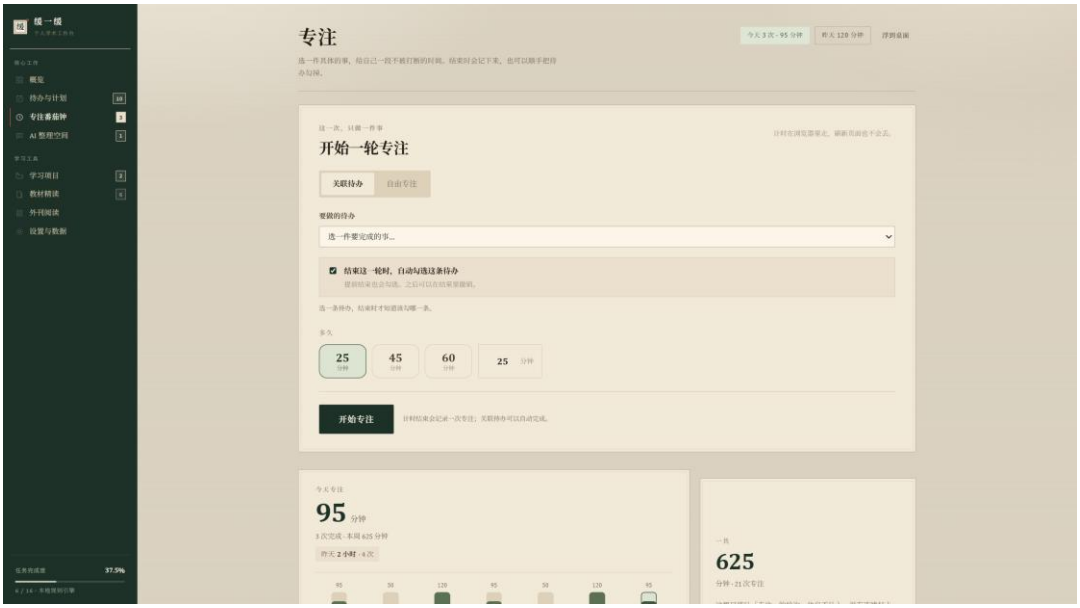


图 2-4　专注：每一轮都绑定一条待办，或一句自由目标

计时开始前必须先选「这一轮做什么」；结束时写入的是实际经过的秒数，而不是计划时长。中途暂停、刷新页面、关掉窗口都不会丢，回来接着计。



图 2-5　教材精读：PDF 拆成章与节，逐节生成讲义

PDF 在浏览器里提取文字与书签，拆成章、节两级；每一节都能生成讲义、对照原文、就这一节提问。讲义会明确标注是本地规则整理还是模型生成。

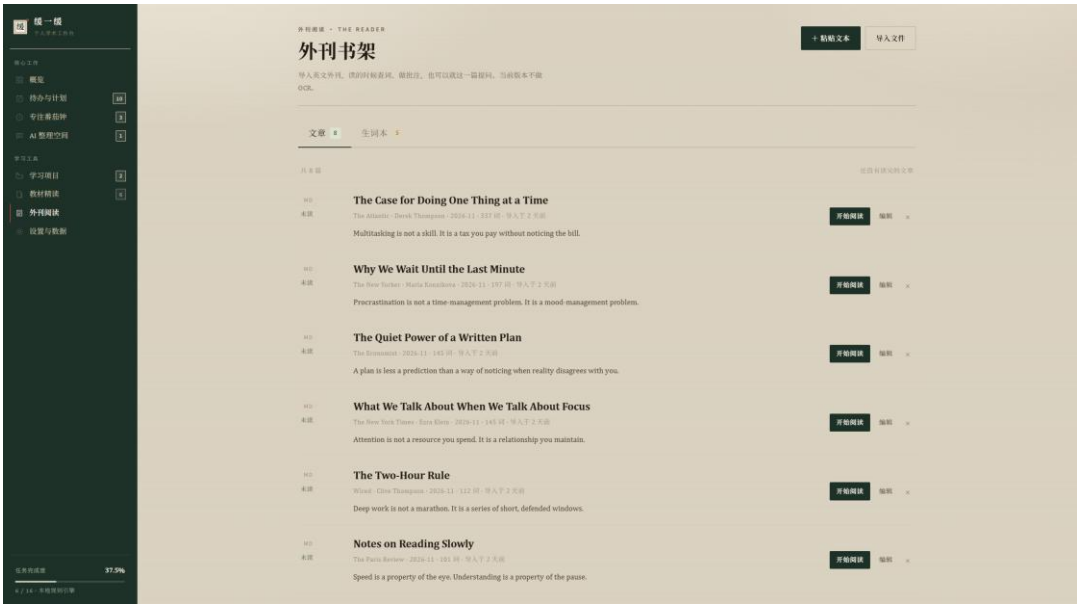


图 2-6　外刊阅读：查词、高亮、批注，生词进生词本

外刊按篇管理，生词与批注跟着文章走；选中一个词就能查，查过的词自动进生词本，不必再开一个背单词 App。



图 2-7　AI 整理空间：先梳理，再谈行动

默认先出「问题梳理卡」——把你说的、没定的、可能的方向分开摆好；只有当你表示想往下走，才生成「行动方案卡」，且必须勾选确认才会变成待办。

2.5 一句话怎么变成一条待办

这条链路是整个作品的骨架，也是「AI 不擅自写数据」的落地方式：

1. 你说一句话，服务端记录原文，不做任何改写。
2. 规则引擎（或你配置的模型）生成「问题梳理卡」：已表达、还没定、可能的方向、下一步。
3. 梳理卡是只读的；你可以直接改文字，改过会标记「用户改过」。
4. 你表示想往下走，才生成「行动方案卡」；每一条都能改标题、改日期、删掉。
5. 你勾选其中几条并确认，服务端才写入 tasks/；任务带 sourceConversationId 与 sourceProposalId，随时能回溯到原对话。
6. 重复确认不会重复建任务：第二次确认会如实返回「已跳过」，而不是再建一遍。

三 核心亮点：六个不那么显眼、但决定成败的设计

3.1 番茄钟绑在一条具体的待办上

多数番茄钟只记「25 分钟」，这里要求先选对象：一条未完成待办，或者一句自由目标。带来的差别不是界面多一个下拉框，而是三个月后你能问出「《电路》第三章我花了多久」「背单词平均每次多久」这种问题。

实现上，进行中的计时只有一份（focus-active.json），网页、桌面浮窗、安卓端读写的是同一个状态，所以关掉窗口再打开不会重新计时；断网时结束的会话会先落在本地队列，联网后补交，按实际结束时间入库。

3.2 教材：拆章、逐节读、就这一节提问

教材不是「上传一个 PDF 然后全文问答」。PDF.js 在浏览器里抽出文字与书签，按书签拆成章、节两级；每一节独立生成讲义，独立提问，回答只引用这一节的原文。阅读位置按字符偏移保存，换设备、换字号都能回到同一句话。

扫描版教材走 Windows 自带 OCR，识别失败会明确说「这一页没读出文字」，而不是编一段看起来像讲义的文字。

3.3 两段式确认：先梳理，再行动

把「聊清楚」和「去执行」分成两步，是产品上最重要的一个决定。直接给方案，用户会被推着走；只给共情，用户什么也拿不到。所以默认只出梳理卡，梳理卡里连「建议」都写得很克制；当用户明确说「帮我排一下」，才出现方案卡，并且方案卡默认一条都不勾选。

3.4 模型自己配：多供应商、多模型、只存本机

应用不预置任何模型，也不内置 Key。第一次打开是规则引擎，功能完整可用，界面会如实标注「这句是本地规则整理，不是模型生成」。要接真模型，在设置里自己加：供应商名称、接口类型（OpenAI 兼容 / Ollama）、Base URL、Key、模型列表，可以加多家、多家里的多个模型。

每条对话还能单独指定模型（modelRef），换模型不会影响历史消息。Key 只写进本机 data/settings.json，接口只回后四位；点「测试连通」会真的发一次最小请求，把模型列表拉回来验证，而不是只看格式对不对。

3.5 浮窗与主程序在同一个进程里

「浮到桌面」曾经是最容易翻车的地方：早期实现由服务端 spawn 一个独立的 Electron 进程，在源码环境能用，装成 exe 之后必然失败——打包目录里没有 node_modules/electron，spawn 自己的可执行文件也只会再开一个主窗口。

现在的做法是把浮窗逻辑抽成 desktop/float-window.js，由主进程在同一个 Electron 里开第二个窗口；主进程另开一条只监听 127.0.0.1、带一次性令牌的内部控制通道给服务端调用。用户点「浮到桌面」不需要再装任何东西，也不会多出一份进程内存。

3.6 服务端零第三方依赖

server/ 下 require 的全是 Node 内置模块，package.json 里没有 dependencies 字段。这意味着十年后拿到源码，装个 Node 就能跑，不需要面对锁文件、镜像源和供应链风险；评审想看代码，也不需要先 npm install。

代价是真实的：没有 ORM、没有框架、没有校验库，参数校验、路由匹配、静态文件缓存都要自己写。这部分多花了大约三成时间，换来的是可读、可迁移、可审计。

3.7 三端独立，但把「将来互通」写进了契约

本期三端数据完全独立，是刻意的：先保证每一端都能单独用，不被同步问题拖住。但代码里已经留下 sync-contract：导出信封带实例身份、记录版本号与墓碑标记，字段只增不改。将来做同步时，不需要回头改历史数据。

四 码道应用说明：先给约束，再让 AI 写，用测试收口

这个项目从第一行代码起就是 AI 辅助开发的。但用法不是「描述需求 → 收代码」，而是固定的三段式：我写约束，AI 出实现，我用断言验收。真正决定质量的是第三段——没有断言，前两段只是在赌。

> 我写约束（PRD 条款 / 边界条件 / 不许做什么）→ AI 出实现（一次可以给很多文件）→ 我跑自测断言（跑绿才算完）

4.1 三类典型用法

用法	具体怎么做	效果
架构落地	先把分层写死（server.js 只做校验与编排、db.js 是唯一持久化入口、models.js 管字段），再让 AI 往骨架里填。	3.7 万行代码没有出现「到处直接读文件」的腐烂。
增量重构	每次只改一个明确边界，改完立刻跑三套自测（接口 / AI / 前端）。	一次能改动十几个文件而不破坏已有行为。
排错	带着完整报错栈、复现条件、已排除的可能去问，而不是「报错了帮我看看」。	下面三个案例都在一轮内定位。

4.2 三个真实案例

案例一：全新数据目录启动时，并发写设置文件报 EPERM

- 现象：桌面版第一次启动，服务端返回 500，日志里是 Windows EPERM（文件被占用）。
- 我给的约束：只改 db.getSettings() 的落盘时机，不许动调用方，不许引入锁文件。
- 定位：AI 指出 getSettings() 每次调用都会「迁移旧配置 → 写临时文件 → rename」，而桌面版启动时主进程与子进程几乎同时调用它，两个临时文件撞在一起。
- 修法：迁移只在内存里做，落盘挪到 ensureDirs() 里串行执行一次。
- 验证：删掉数据目录重建，连开 5 次不再报错。

案例二：批量删除对话返回 500（SAFE_DELETE_BULK_CONFIRM_REQUIRED）

- 现象：批量删除直接 500，前端只显示「操作失败」。
- 我给的约束：不许把内部错误码透给用户；删掉的东西要能找回。
- 修法：fsp.rm 换成 moveToTrash()，文件挪进 data/.trash/；5xx 响应统一脱敏。

- 验证：自测里加一条「删除后原文件仍在 .trash 里」的断言，防止以后改回去。

案例三：桌面版点「浮到桌面」却提示要先装 Electron

- 现象：装好的 exe 里点浮窗，弹出「请先 npm install --save-dev electron」。
- 我给的约束：不许让用户为浮窗再装任何东西；不许起第二个 Electron 进程（内存翻倍）。
- 定位：三个原因叠在一起——① 判定逻辑去找 node_modules/electron，打包后不存在；② 打包清单只写了 desktop/main.js，浮窗文件根本没进包；③ 就算找到，spawn 自己的 exe 也只会再开一个主窗口。
- 修法：浮窗逻辑抽成 desktop/float-window.js，由主进程在同一个 Electron 里开第二个窗口；主进程开一条只监听 127.0.0.1、带一次性令牌的通道给服务端用。
- 验证：开发态日志三步（已新开窗口 → 页面已加载 → 第二次叫回已有窗口）；打包版冒烟 4/4 通过，浮窗项返回「进程内开窗」。

4.3 沉淀下来的五条方法

1. 把「不要什么」写进测试，比写「要什么」更能约束 AI 的行为边界。
2. 报错要给全：完整堆栈 + 复现步骤 + 已排除项，比反复描述现象省三轮。
3. 一次只放开一个边界：改数据层就不顺手改界面，否则测试挂了不知道是谁的锅。
4. 让 AI 先复述约束再动手：理解偏了在第一步就能发现，而不是在第十个文件之后。
5. 文档与代码同源：说明里的每条约束都能在自测里找到对应断言，避免「文档说得好、代码没做」。

4.4 明确没有交给 AI 的部分

- 产品判断：哪些功能进这一版、哪些砍掉（章节知识图谱、错题整理都砍了），由人定；
- 视觉方向：纸页与书房这套低饱和、无排行榜、无打卡的调性，是人工反复调出来的；
- 数据取舍：旧记录怎么迁移、哪些字段能删、哪些只能加墓碑，人工决策；
- 最终验收：所有外发产物（exe、APK）都在真机或模拟器上跑过一遍才留下。

4.5 过程记录在哪

codearts_proof/案例过程记录.md 与本节三个案例逐条对应：约束、AI 方案、被否决的版本、断言与验证输出；screenshots/ 放原始对话截图。

五 工程数据与验证结果

5.1 代码规模

部分	行数	说明
服务端 server/	16,270	23 个模块，仅使用 Node 内置模块
前端 public/	20,106	原生 JS + CSS，无框架、无构建步骤
桌面壳 desktop/	990	主进程、浮窗模块、预加载脚本
合计	37,366	不含第三方库与 PDF.js

5.2 自测：765 项断言

套件	项数	覆盖内容	结果
接口自测 server/selftest.js	388	REST 接口、迁移、回收站、权限与边界、坏输入	388 / 388 通过
AI 自测 server/agent-selftest.js	55	三引擎切换、工具调用、契约字段、导出信封	55 / 55 通过
前端自测 server/uitest.js	322	各视图渲染、交互、浮窗外壳、文案约束	322 / 322 通过

三套自测都跑在独立的数据目录上（WB_DATA_DIR 指向临时目录），不碰使用者真实数据。其中接口自测包含一条迁移断言，需要目录里存在第一版的旧记录，因此验证时放入了一条真实格式的 entries 记录，迁移后再断言档案可追溯。

断言里相当一部分是「不许做什么」的反向约束，例如：欢迎区不许出现催促文案；本地规则引擎整理的内容不许标成「模型生成」；AI 不许在用户没确认时建待办；删对话不许真删文件。这些一旦写进测试，后续无论怎么重构都绕不过去。

5.3 产物与实测

产物	大小	怎么验证的	结果
缓一缓-2.0.0-便携版.exe	107 MB	直接运行，走完冒烟脚本（启动、接口、浮窗、退出清理）	4 / 4 通过
缓一缓-2.0.0-安装版.exe	118 MB	安装到默认目录后启动，检查数据目录与快捷方式	正常
缓一缓-2.0.0-安卓版.apk	65 MB	模拟器安装 → 启动 → 写入任务 → 强杀重启	数据仍在
缓一缓-2.0.0-源码.zip（不含 node_modules）	2.5 MB	解压后直接 node server/server.js	无需安装依赖

桌面版首次启动需 3–5 秒拉起内置服务；安卓版首次启动需 10–20 秒拉起内嵌 Node 运行时，属预期行为。

5.4 已知限制

限制	说明
没有购买代码签名证书	Windows 首次运行会提示「未知发布者」，点「更多信息 → 仍要运行」即可。
扫描版教材依赖 Windows OCR	公式、多栏排版会失真；识别失败会明确告知，不会编造内容。
安卓版首次启动较慢	需要先拉起内嵌的 Node 运行时；被系统省电策略限制时可能更久。
三端尚未互通	本期各自独立；同步契约已在代码中预留，未实现同步逻辑。
远程模型会把对话原文发出去	设置页对此有明确说明；介意的话用本地规则引擎或 Ollama。

六 交付物与运行方式

6.1 提交目录

目录	内容
ppt/	本说明文档（Word 与同版 PDF）
project/	可演示的完整产物：Windows 便携版与安装版、安卓 APK、源码包
readme/	环境依赖、一键安装脚本、部署步骤、开源组件、码道应用总结
video/	三分钟演示脚本与录制清单
codearts_proof/	与第四章案例对应的过程记录、Prompt 与验证输出

6.2 三条运行路径

路径一：桌面版（评审最快看到完整功能）

1. 双击 project/缓一缓-2.0.0-便携版.exe（便携版，不写注册表，无需安装）。
2. 首次启动等 3–5 秒，浏览器内核与本地服务就绪后进入工作台。
3. 数据落在 %APPDATA%\huan-yihuan-desktop\data，与源码版、安卓版完全隔离。
4. 想看浮窗：待办页或专注页右上角点「浮到桌面」，可吸边收成小球。

路径二：安卓版

1. 把 project/缓一缓-2.0.0-安卓版.apk 传到手机安装（允许安装未知来源应用）。
2. 首次启动等 10–20 秒：App 内要先拉起 Node 运行时再加载页面。
3. 数据写在 App 私有目录，卸载即清空；不联网也能完整使用。

路径三：源码

1. 确认 Node.js ≥ 18（推荐 20 / 22 LTS）。
2. 在项目根目录运行 node server/server.js，浏览器打开 http://127.0.0.1:5180。
3. 不需要 npm install：服务端没有第三方依赖。

6.3 首次配置模型（约 3 分钟）

步骤	在哪	做什么
1	打开应用	直接能用，此时是规则引擎，页面会标注内容来源。
2	侧栏底部 → 设置与数据	添加供应商：名称、接口类型、Base URL、API Key。
3	同一页 → 测试连通	真的发一次最小请求，把模型列表拉回来验证。
4	选一个默认模型	之后所有对话默认用它；单条对话还能再换。
5	想用本地模型	装 Ollama → 拉一个模型 → 供应商类型选 Ollama，无需 Key。

6.4 数据在哪、怎么备份

形态	位置
网页版 / 源码版	项目下的 data/
桌面版	%APPDATA%\huan-yihuan-desktop\data
安卓版	App 私有目录（app.datadir()/data）

备份就是把整个 data/ 目录复制走，恢复就是放回去。没有数据库，没有专有导出格式，JSON 用记事本就能读；教材 PDF 原文存在 data/textbooks/ 下。

删除是软删除：对话、任务、档案被删后进入 data/.trash/，需要时可以找回。这也是自测里的一条断言——「删对话不许真删文件」。

— 文档完 —

缓一缓 · 个人学术工作台　v2.0.0　2026 年 11 月